HPC Solution Autumn End Semester Examination-2025

1. Answer the following questions.

(a) Consider an instruction pipeline with 5 stages that take 7 nsec, 4 nsc, 3 nsec, 8 nsec, and 5 nsec respectively. The delay of an inter-stage register stage of the pipeline is 2 nsec. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation? (1 marks)

Answer 1. (a) In **non-pipelined** execution, total time per instruction

= 7 + 4 + 3 + 8 + 5

= 27 ns.

In **pipelined** execution, the clock cycle time is set by the slowest stage plus the register delay:

max(7, 4, 3, 8, 5) + 2 ns

= 10 ns.

$$\text{Speedup} = \frac{\text{Time}_{\text{non-pipeline}}}{\text{Time}_{\text{pipeline}}} = \frac{27\ \text{ns}}{10\ \text{ns}} \approx 2.7$$

Thus, under ideal conditions, the pipeline is approximately 2.7 times faster than the non-pipelined implementation for long instruction sequences.

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(b) Write down the corresponding assembly language MIPS instruction for the following machine code given in the hexadecimal form:

018E7822

016D6020

The opcode of load, store, add and sub is 35, 43, 32 and 34 in integers.

(1 marks)

Answer 1. (b) **018E7822**

- Hex to binary: **0000 0001 1000 1110 0111 1000 0010 0010**
- Opcode: first 6 bits (**000000**) → R-type
- Registers (next 15 bits 5 bits each):
  rs (**000011**=3), rt (**10001**=17), rd **(11001**=25).
  ANS: SUB R25,R3,R17

016D6020

- Hex to binary: **0000 0001 0110 1101 0110 0000 0010 0000**
- Opcode: (**000000**) → R-type.
- Registers: rs (**001101**=13), rt (**01101**=13), rd (**01100**=12).
- funct **(100000**=32) ⇒ add.
  ANS: ADD R12, R13, R13

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(c) Consider two processors P1 and P2 executing the same instruction set. Assume that under identical conditions, for the same input, a program running on P2 takes 25% less time but incurs 30% more CPI (clock cycles per instruction) as compared to the program running on P1.If the clock frequency of P1 is 2 GHz, then calculate the clock frequency of P2 (in GHz)? (1 marks)

Answer 1.(c) Execution time for a given program:

$$T = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Frequency}}$$

For P1: $T_1 = \frac{IC \cdot \text{CPI}_1}{f_1}$

For P2: $T_2 = \frac{IC \cdot \text{CPI}_2}{f_2}$

Using $T_2 = 0.75\,T_1$ and $\text{CPI}_2 = 1.3\,\text{CPI}_1$, and dividing the two equations:

$$\frac{T_2}{T_1} = \frac{\text{CPI}_2}{\text{CPI}_1} \cdot \frac{f_1}{f_2}$$

$$0.75 = 1.3 \cdot \frac{f_1}{f_2}$$

$$\frac{f_2}{f_1} = \frac{1.3}{0.75}$$

$$f_2 = f_1 \cdot \frac{1.3}{0.75} = 2\,\text{GHz} \cdot \frac{1.3}{0.75} = 2\,\text{GHz} \cdot 1.733\ldots \approx 3.47\,\text{GHz}$$

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(d) What hardware implementation exists in MIPS to avoid structural hazard while referencing memory and register? (1 marks)

Answer 1.(d) Split Cache (Harvard Architecture):

- MIPS uses physically separate instruction and data memories (split caches), allowing concurrent access to instructions and data.
- This avoids conflicts when one pipeline stage fetches instructions while another accesses or writes data, enabling parallelism and preventing stalling due to shared memory resources.

- Half-Clock Cycle Protocol for Registers:
  - The MIPS register file is designed to support writing during the first half of the clock cycle and reading during the second half.
  - This protocol allows an instruction to write to a register and another to read from it without timing conflicts in the same clock cycle, eliminating structural hazards in register access.

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(e) What are the two ways of handling cache read miss operation? (1 marks)

Answer 1.(e)Write-Allocate (Fetch-On-Read):

- On a cache read miss, the requested data block is fetched from memory into the cache, and then the read request is fulfilled. This approach keeps frequently accessed data close to the processor, improving hit rates for subsequent accesses.

- No-Write-Allocate (Read-Through):
  - On a cache read miss, the data is read directly from main memory without loading it into the cache. Some read-through cache designs automatically populate the cache on a miss, but in classic no-write-allocate for reads, the cache may bypass allocation on a miss depending on system needs.

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(f) Calculate the percentage of miss prediction accuracy using 2-bit dynamic branch prediction scheme for the following outcome of a branch:

T, NT, T, T, T, NT, NT, NT, T, T, NT, T, NT, T, NT, NT

Assume the initial prediction is **strongly NOT taken**. (1 marks)

ANSWER 1 (f):

- NT: Strongly Not Taken (00)
- nt: Weakly Not Taken (01)
- tT: Weakly Taken (10)
- T: Strongly Taken (11)

Step	Actual	State start	Predicted	Mispredict?	New state
1	T	NT	NT	Yes	nt
2	NT	nt	NT	No	NT
3	T	NT	NT	Yes	nt
4	T	nt	NT	Yes	tT
5	T	tT	T	No	T
6	NT	T	T	Yes	tT
7	NT	tT	T	Yes	nt
8	NT	nt	NT	No	NT
9	T	NT	NT	Yes	nt
10	T	nt	NT	Yes	tT
11	NT	tT	T	Yes	nt
12	T	nt	NT	Yes	tT
13	NT	tT	T	Yes	nt
14	T	nt	NT	Yes	tT
15	NT	tT	T	Yes	nt
16	NT	nt	NT	No	NT

For the given 2-bit branch prediction sequence:

- Number of misses (mispredictions): 11
- Number of hits (correct prediction

Mispredictions: 11 out of 16 (68.75%). Accuracy: 31.25%.

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(g) In which stage of the scoreboard approach, the output dependency and anti-dependency have been resolved? (1 marks)

ANSWER 1 (g):In the scoreboard approach, output dependency (WAW) is resolved during the Issue stage, and anti-dependency (WAR) is resolved during the Write Result (WB) stage.

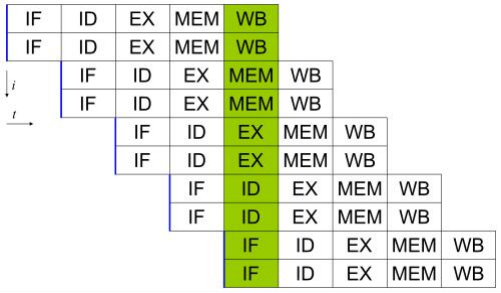
- Output dependency (WAW): Checked and resolved when instructions are issued, ensuring no two instructions are allowed to write to the same destination register simultaneously.
- Anti-dependency (WAR): Detected and resolved before writing results back in the Write Result stage, by stalling the instruction if any active instruction still needs to read the register.

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

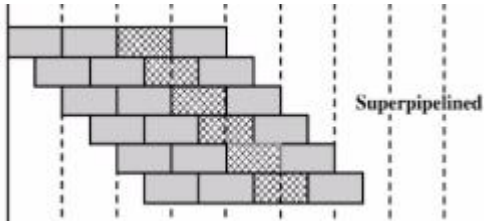
(h) Distinguish between super-scalar and super-pipeline processor. (1 marks)

ANSWER 1 (h):

Aspect	Superscalar Processor	Super-pipeline Processor
Definition	Issues multiple instructions per clock cycle	Increases pipeline stages for finer division
CPI	CPI can be less than 1	CPI approaches 1
Diagram Point	Shows multiple parallel pipelines	Shows deeper, sequential pipeline stages



Super Scalar



Super pipelined

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(i) In a real time system, it's observed that a CPU generate 200 memory references out of which 10 misses in L1 cache, 6 misses in L2 cache. The miss penalty for L2 cache to memory is 200 clock cycles. The hit time for L2 cache and L1 cache are 5 clock cycles and 1 clock cycle respectively. What is the local miss rate for L2 cache and AMAT?

(1 marks)

ANSWER 1 (i):

Calculation of Local Miss Rate for L2 Cache

- Number of accesses to L2 cache: This equals the number of misses in L1 cache = 10
- Number of misses in L2 cache: 6

Local Miss Rate (L2)=6/10=0.6 or 60%

Calculation of AMAT (Average Memory Access Time)

Use the formula for a two-level cache:

$$\text{AMAT} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2})$$

Given:

- $\text{Hit time}_{L1} = 1$
- $\text{Miss rate}_{L1} = \frac{10}{200} = 0.05$
- $\text{Hit time}_{L2} = 5$
- $\text{Miss rate}_{L2} = 0.6$
- $\text{Miss penalty}_{L2} = 200$

$$\begin{aligned}
\text{AMAT} &= 1 + 0.05 \times (5 + 0.6 \times 200) \\
&= 1 + 0.05 \times (5 + 120) \\
&= 1 + 0.05 \times 125 \\
&= 1 + 6.25 \\
&= 7.25 \text{ clock cycles}
\end{aligned}$$

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

(j) Explain Compulsory, Capacity and Conflict misses in Cache.

(1 marks)

ANSWER 1 (j):

- Compulsory miss: Occurs the first time a memory block is accessed because it is not yet present in the cache.
- Capacity miss: Happens when the cache cannot hold all the needed blocks, resulting in evictions due to limited cache size. Happens in Fully Associative Cache.
- Conflict miss: Arises when multiple blocks compete for the same cache location/set, causing misses even when total cache space is sufficient. Happens in Set Associative Cache.

Evaluation Scheme: Full mark for correct answer. Zero mark for incorrect answer

SECTION-B

2. (a) Derive the overall speed up gained by Amdahl's law. [5]

Suppose a processor spends 62% of its time in memory operations (both I- cache and D-cache access together) when running a given program A and its manufacturers make a change that improves its performance on memory access by a factor of 6. The program originally took 212 sec to execute. What is the speed up from the old system to the new system? What fraction of its execution time does the new system spend in memory operations?

Solution 2 (a) Let the total original execution time of a program be T_{orig}. Split the program into two parts:

A fraction **p** of the execution time that **can be parallelized**.

The remaining fraction **(1−p)** that is **inherently sequential** (cannot be parallelized).

Assume the parallelizable part is speed up by a factor **S**. The sequential part remains unchanged.

Therefore the new execution time $T_{new} = T_{orig}. (1-p) + T_{orig} . (p/S)$

The parallel portion takes 1/S of its original time.

Overall Speedup = $T_{orig}/T_{new} = T_{orig} / (T_{orig}. (1-p) + T_{orig} . (p/S))$

$= 1/((1-p) + (p/S))$

Given

Fraction of time spent in memory operations p = 0.62

Speedup of memory subsystem S=6

Original execution time: T_{old} = 212 sec

1. Overall Speedup Using Amdahl's Law

Amdahl's Law:

$$\text{Speedup} = 1/((1-p) + (p/S))$$

$$= 1/((1-0.62)+(0.62/6))$$

$$= 1/(0.38 + 0.10333) = 1/0.48333 = 2.0689$$

New Execution time $T_{new} = T_{old}$ /speedup = 212/2.0689 = 102.4699

Fraction of Time in Memory Operations on the New System

New time spent in memory:

$$T_{new(Memory)} = (p.T_{old})/6 = (0.62 \text{ X } 212)/6 = 21.9$$

Fraction of new execution time = $T_{new(Memory)}/T_{new}$ = 21.9/102.4699 = 0.2137

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

(b) Find out the total no of clock cycles required to execute the following instructions without and with operand forwarding and draw the **time and space diagram for both the cases**? [5]

```
LD R11, 50(R2)
DADDIU R11, R11, #10
SD R11, 40(R2)
DADDIU R5, R2, #4
DSUB R4,R3,R5
```

Solution 2. (b)

Assumptions Taken

Standard MIPS-like 5-stage pipeline: IF → ID → EX → MEM → WB.

Writes happen in the 1st half of the cycle. Reads happen in the 2nd half of the same cycle.

Forwarding paths exist from EX/MEM and MEM/WB to EX when "with forwarding".

Without Forwarding 15 cycles

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12	CC13	CC14	CC15
LD R11,50(R2)	IF	ID	EX	MEM	WB										
DADDIU R11,R11,#10		IF	STALL	STALL	ID	EX	MEM	WB							
SD R11,40(R2)					IF	STALL	STALL	ID	EX	MEM					
DADDIU R5,R2,#4								IF	ID	EX	MEM	WB			
DSUB R4,R3,R5									IF	STALL	STALL	ID	EX	MEM	WB

With Forwarding 10 cycles

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10
LD R11,50(R2)	IF	ID	EX	MEM	WB					
DADDIU R11,R11,#10		IF	ID	STALL	EX	MEM	WB			
SD R11,40(R2)				IF	ID	EX	MEM			
DADDIU R5,R2,#4					IF	ID	EX	MEM	WB	
DSUB R4,R3,R5						IF	ID	EX	MEM	WB

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

3. (a) Explain how reservation stations and Common Data Bus(CDB) used in the Tomasulo's approach are useful in resolving the WAW and WAR hazard with the following example for each hazard.[Assumption: [5]

Initially all the registers are available, i.e., not waiting for any functional units. The number of functional units is also sufficient enough to accommodate all the instructions given.]

Find out the total number of clock pulse required to execute the following instructions using Tomasulo's approach.

ADD Fl, F2, F3

STORE Fl, 5(R4)

ADD F1, F5, F6

SUB F7, F1, F8

STORE Fl, 6(R5)

ADD F1, F9, F10

Solution (a)

Reservation Stations and CDB in Tomasulo's Approach

Reservation Stations: These are buffers that hold instructions waiting to execute with their operands. They allow instructions to issue out-of-order while maintaining data dependencies and avoiding hazards.

Common Data Bus (CDB): The CDB broadcasts the result of functional units to all registers and reservation stations, enabling register and operand updates simultaneously.

Resolving Hazards:

WAW Hazard (Write-After-Write): Occurs if two instructions write to the same register out-of-order. Reservation stations track issued writes, so only the last write updates the register when ready, preventing premature writes.

WAR Hazard (Write-After-Read): Occurs if an earlier read depends on a later write. Since operands are read from reservation stations when ready, an instruction won't read a register until its value is available, avoiding WAR hazards.

	Op	Busy	Vj	Vk	Qj	Qk	A
1. ADD F1,F2, F3	ADD	YES(1) No(4)	[F2]	[F3]	-	-	-
2.STORE F1, 5(R4)	Store		R4			ADD1	5+[R4]
3.ADD F1,F5, F6	ADD	YES(3)	[F5]	[F6]			
4.SUB F7,F1, F8	SUB	YES(4)		[F8]	ADD2		
5. STORE F1, 6(R5)	Store		R5			ADD2	6+[R5]
6.ADD F1,F9, F10	ADD	YES(6)	[F9]	[F10]			

Register Status Table

F1	F2	F3	F4	F5	F6	F7	F8	F9	F10

~~ADD1(1)~~ ~~ADD2(3)~~ ADD3(6)						SUB(4)			

1. Instruction / timing table

Instruction	Issue	Execution	WB
ADD F1, F2, F3	1	2–3	4
STORE F1, 5(R4)	2	3–4	–
ADD F1, F5, F6	3	4–5	6
SUB F7, F1, F8	4	7–8	9
STORE F1, 6(R5)	5	10–11	12
ADD F1, F9, F10	6	7–8	9

Assumption: All are taking 2 CCs in the Execution stage.

As each unit is taking the dependent value from the CDB instead of the registers, WAW and WAR hazards are eliminated even though we are executing the instructions out of order. Eg. 5th no instruction will read the value from CDB when instruction no 3 is putting the result of [F5]+[F6].

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

(b) Explain loop unrolling and perform loop unrolling for the following code fragment [5]

represented in high level language-

```
for ( i=0; i<=100; i++)
{
    a[i] = a[i] - 4;
}
```

This code subtracts a scalar to a vector. Here each element in the array is of type double, occupying 8 bytes. In the pipeline branch condition is checked in the Execution stage. Ignore the delays introduced by the branch instruction.

Assume the following latency in the example:

Instruction producing the result	**Instruction using the result**	**Latency in clock cycles**
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load Double	FP ALU op	1
Load Double	Store double	0
Integer operation	Integer op	0

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

Solution (b) Loop Unrolling is a scheme for increasing the number of instructions relative to the branch and overhead instructions. Unrolling simply replicates the loop body multiple times, adjusting the loop termination code.

```
1.fld f0, 0(R1) ; a[i]
2.fsub.d f4, f0, f2 ; a[i] - 4, [F2] contains 4
3.fsd f4, 0(R1)

4.fld   f6,  -8(R1)     ; a[i+1]
5.fsub.d f8, f6, f2
6.fsd   f8,  -8(R1)

7.fld   f10, -16(R1)    ; a[i+2]
8.fadd.d f12, f10, f2
9.fsd   f12, -16(R1)

10.fld   f14, -24(R1)    ; a[i+3]
11.fsub.d f16,f14,f2
12.fsd   f16,-24(R1)

13.addi  R1, R1, -32     ; move to next block of 4
14.bne   R1, R2, Loop
```

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

4. (a) For a given machine, following are the different type of instructions and their corresponding clock cycles:- [5]

LOAD	5
STORE	4
ALU	4
BRANCH	3
JUMP	3

If a program has 50% ALU instructions, 15% LOAD instructions, 25% STORE instructions, 8% BRABCH instructions and 2% JUMP instructions, then evaluate the CPI overall. If the same program has 2000 instructions and the speed of the computer is 2.4GHz, calculate the time taken to execute the program.

Solution 4 (a)

4.a. $CPI = \sum(\text{fraction} \times \text{cycle})$

$= 0.5(4) + 0.15(5) + 0.25(4) + 0.08(3) + 0.0$

$= 4.05$ cycles.

clock period $= 1/2.4 \times 10^9$ s

Execution time $= 2000 \times 4.05 \times \frac{1}{2.4 \times 10^9}$

$= 8100 \times \frac{1}{2.4 \times 10^9} = 3.375 \times 10^{-6}$ sec

$= 3.375\ \mu s$

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

(b) How a centralized shared memory system is different from distributed shared memory system. Explain with a suitable diagram.

Solution 4 (b)

(b) Difference Between Centralized Shared Memory System and Distributed Shared Memory System – With Diagram

Shared memory systems allow multiple processors to access a common memory space for communication. However, the way memory is organized leads to two major types:

---

1. Centralized Shared Memory System (Uniform Memory Access – UMA)

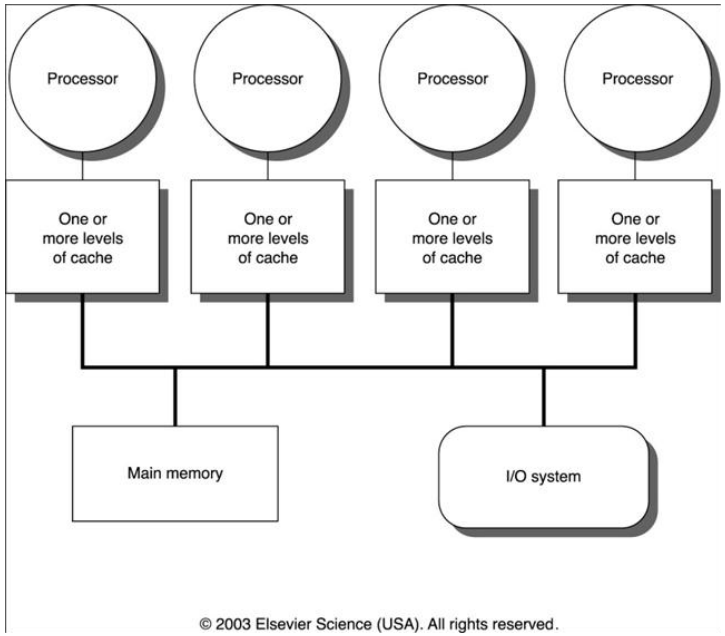
Definition

A centralized shared memory system contains **a single, large main memory** that is shared by **all processors**.

This memory is connected to processors through a **common bus, crossbar, or interconnection network**.

Key Features

1. **Single shared memory module** for all processors.
2. **Uniform access time** for every processor because memory is centralized (UMA).
3. **Easier to design and program** because memory behaves like one big space.
4. Memory bandwidth can become a **bottleneck** when many processors simultaneously access memory.
5. Suitable for small to medium multiprocessor systems.



© 2003 Elsevier Science (USA). All rights reserved.

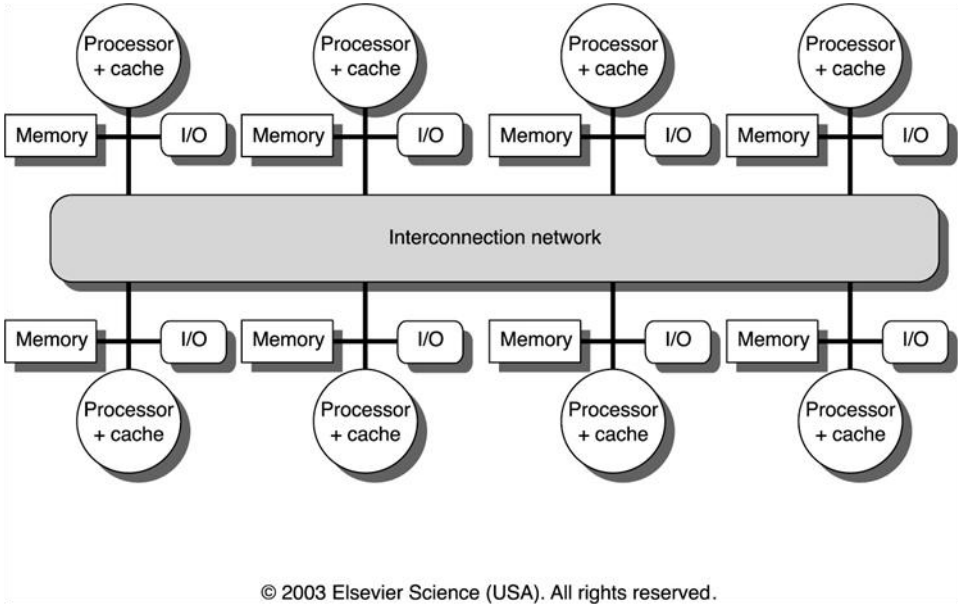
2. Distributed Shared Memory System (NUMA – Non-uniform Memory Access)

Definition

A distributed shared memory (DSM) system has **multiple memory modules**, each **physically located near a processor**.
Even though memory is distributed physically, it is **logically shared**, meaning any processor can access any memory.

Key Features

1. Memory is **divided** into modules, each attached to a specific processor.
2. Accessing **local memory is faster**; accessing **remote memory is slower** → non-uniform access time (NUMA).
3. Scales better to large systems since no single memory module becomes a bottleneck.
3. Scales better to large systems since no single memory module becomes a bottleneck.
4. More complex memory consistency and communication mechanisms.
5. Suitable for large multiprocessor or cluster-based systems.



© 2003 Elsevier Science (USA). All rights reserved.

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

5. (a) Assume a fully associative write back cache with many cache entries that starts empty and block size is 4 Bytes and word length is 1 byte. Below is a sequence of memory operations (the address is in square brackets). [5]

Write Mem [2500]

Read Mem [2400]

Write Mem [3000]

Read Mem [2000]

Read Mem [6000]

Write Mem [3000]

Write Mem [2000]

Read Mem [3000]

Write Mem [3000]

What are the numbers of hits and misses (inclusive read and write) when using no-write allocate and write allocate?

Solution a).

(a) Assume a fully associative write back cache with many cache entries that starts empty and block size is 4 Bytes and word length is 1 byte. Below is a sequence of memory operations (the address is in square brackets).

Write Mem [2500]	Miss	Miss
Read Mem [2400]	Miss	Miss
Write Mem [3000]	Miss	Miss
Read Mem [2000]	Miss	Miss
Read Mem [6000]	Miss	Miss
Write Mem [3000]	Hit	Miss
Write Mem [2000]	Hit	Hit
Read Mem [3000]	Hit	Miss
Write Mem [3000]	Hit	Hit

(write allocate) 4 Hit + 5 Miss

(write no allocate) 2 Hit + 7 Miss

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

(b) What are the various optimizations techniques applied for reducing the hit time, cache miss penalty & Reduce Miss Rate? [5]

Solution b)

Ans 5 (b)

1. Techniques to Reduce Hit Time
   - Small and Simple Caches: Using smaller L1 caches reduces access time because fewer entries need to be searched.
   - Virtually Indexed Caches: Allows parallel TLB and cache access, reducing lookup time.
   - Way Prediction: Predicts which way of a set-associative cache will be accessed, reducing average access time.
   - Pipelined Cache Access: Divides cache access into stages to improve speed without increasing clock cycle time.

2. Techniques to Reduce Miss Penalty
   - Critical Word First & Early Restart: Start sending the requested word before the entire block is fetched.
   - Giving Priority to Read Misses: If writes are pending, read requests are serviced first, reducing delay.
   - Multi-level Caches: L2 and L3 caches significantly reduce the penalty of an L1 miss.
   - Sub-blocking: Divides blocks into smaller pieces to avoid fetching the entire block when not needed.

3. Techniques to Reduce Miss Rate
   - Increasing Cache Size: Larger caches store more data and reduce compulsory and capacity misses.
   - Higher Associativity: Moves from direct-mapped to 2-way or 4-way associative caches to reduce conflict misses.
   - Better Replacement Policies: Using LRU or LFU instead of random replacement decreases miss rate.
   - Compiler Optimizations: Loop interchange, blocking, and data locality optimizations improve cache usage.

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

6. (a) Assume two cache memory specifications. A split cache which consists of 16KB instruction cache and 16KB data cache and 32KB unified cache. The cache memory hit time is 2 clock cycles, miss penalty is of 85 clock cycles. [5]

For the above cache memory specifications a simulator showed the following conditions like:

I) 15 miss per 1000 instructions for data cache.

II) 30 miss per 1000 instructions for instruction cache.

III) 45 miss per 1000 instructions for the unified cache.

If there are 35% Load/Store instruction then calculate the miss rate and average memory access time (AMAT) for split and unified cache memory.

Solution a)

Data misses = 15 per 1000 instructions.

Instruction misses = 30 per 1000 instructions.

Unified misses = 45 per 1000 instructions.

Fraction of Load/Store (data accesses) = 0.35 → data accesses per 1000 instr = 0.35×1000 = 350.

Instruction fetches per 1000 instr = 1000.

Hit time = 2 cycles.

Miss penalty = 85 cycles.

Miss rates (per access)

Data miss rate (per data access)
= 15 misses / 350 data accesses
= 15/350=3/70≈0.0428571=4.2857%.

Instruction miss rate (per instruction fetch)
= 30 / 1000 = 0.03 = **3.00%**.

Unified miss rate (per unified cache access)
total accesses per 1000 instructions = 1000 (instr) + 350 (data) = 1350.
unified miss rate = 45 / 1350 = 1/30≈0.0333333=3.3333%.

AMAT (average memory access time)

AMAT formula (per access): AMAT=Hit time+Miss rate×Miss penalty.

Split cache:

AMAT = %instr x (instr hit time + instr miss rate x instr miss penalty) +

%data x (data hit time + data miss rate x data miss penalty)

= 1 X (2+0.03×85)+ 0.35 X (2+(3/70)×85)

=1 X (2+2.55)+0.35 X (2+3.642857)

=4.55+1.975=6.525 cycles per access

Unified cache:

AMAT (per unified access) = 2+(1/30)×85=2+2.833333=4.833333 cycles per access.

cycles per instruction = 1.35×4.833333=6.525 cycles per instruction.

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer

(b) Explain the working and implementation of snoopy based protocol with state diagram in details. [5]

Solution b)

A **snoopy protocol** is a cache-coherence mechanism used in **shared-bus multiprocessor systems**. Each processor has a private cache, and all caches **monitor ("snoop")** the shared bus to maintain coherence.

Working of Snoopy-Based Protocol

Every cache controller continuously **listens (snoops)** to the bus to detect memory operations issued by other processors. Based on what it hears, it updates its own cache state to maintain a consistent memory view across processors.

(a) **Processor Issues a Read or Write**

- If the data is in cache → served locally.
- If not → cache sends a **Bus Read** or **Bus Read-Exclusive** request.

(b) **Other Caches Snooping** : Other caches listen to the bus:

- If they have a copy of requested data, they may **respond**, **invalidate**, or **update** depending on the protocol.

(c) **Maintaining Coherence**

Two outcomes maintain coherence:

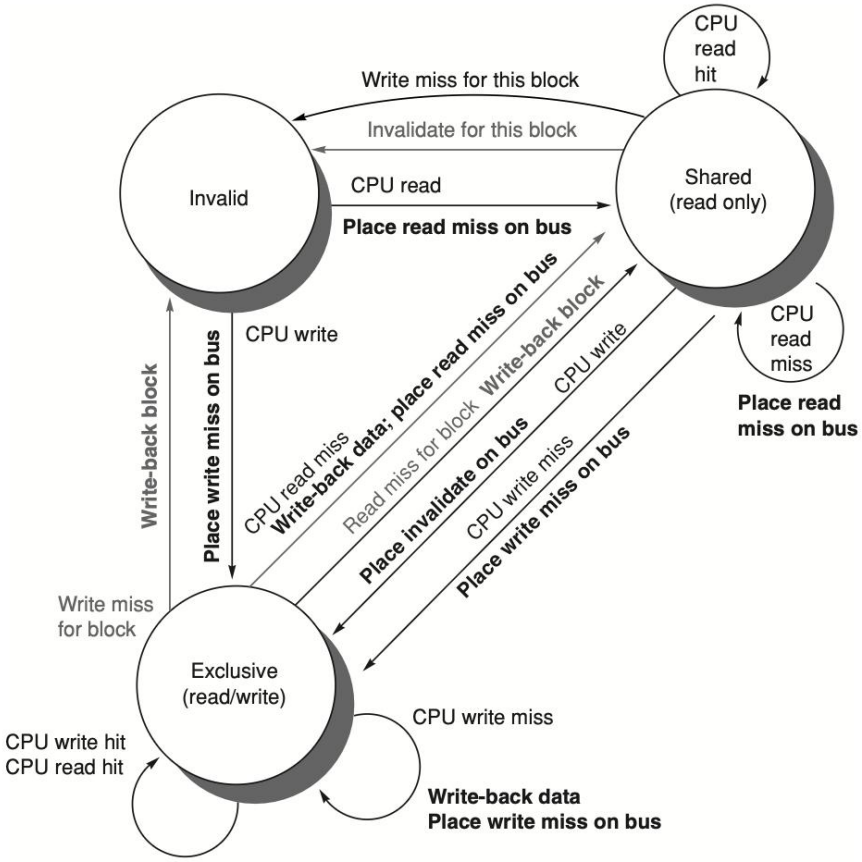
1. **Invalidation:**
   When one processor writes, all other caches invalidate their copies.
2. **Update:**
   When one processor writes, it also pushes the updated value to other caches.

Most snoopy protocols use **invalidations** (MSI, MESI, MOESI).

Snoopy Protocol Architecture

- **Shared Bus:** All caches communicate over it.
- **Cache Controller:** Snoop unit monitors the bus.
- **Directory not needed** → simpler than directory-based protocols.

Each cache line contains **coherence state bits** (MSI or MESI states).



State Diagram

Evaluation Scheme: Full mark for correct answer. Step marking may be awarded for partially correct correct answer